Parameter-Efficient Adaptation of TokenGT using LoRA

Final Project Progress Report

Revak Sai (22b1038) Maloth Vishal Nayak (22b1015)
N Harsha Vardhan(22b0977)

May 9, 2026

Abstract

Graph-structured data appears naturally in many domains such as social networks, molecular graphs, knowledge graphs, biological systems, and recommendation platforms. Traditional graph learning models, especially Graph Neural Networks (GNNs), usually rely on message passing, where every node aggregates information from its local neighborhood. Although this is effective, message passing can struggle with long-range interactions and may suffer from over-smoothing when the network becomes deep.

The Tokenized Graph Transformer (TokenGT) proposes a different approach by treating nodes and edges as tokens and processing them using a standard Transformer encoder. In our project, we studied the TokenGT framework and modified its implementation using Low-Rank Adaptation (LoRA). Instead of fine-tuning all model parameters, LoRA freezes the original model weights and inserts small trainable low-rank matrices into selected linear layers. We applied LoRA to the attention projection layers of the sparse Laplacian-type TokenGT setup and compared different LoRA ranks: 4, 16, 64, 128, and 256.

The full baseline model achieved a test loss of **0.017566**. LoRA did not outperform the fully trained baseline, but it showed a clear rank-dependent improvement. The test loss reduced from **1.34** at rank 4 to **0.06** at rank 256. These results show that higher LoRA rank increases adaptation capacity and helps recover more of the baseline performance. At the same time, the remaining gap suggests that attention-only LoRA is still less expressive than full-parameter training for this synthetic graph learning task.

1 Introduction

Graphs are a natural way to represent relational data. A graph consists of nodes and edges, where nodes represent entities and edges represent relationships between them. Many real-world datasets can be represented as graphs, including social networks, molecular structures, transportation networks, citation networks, and knowledge graphs.

Graph Neural Networks (GNNs) have been widely used for graph representation learning. In a typical GNN, each node updates its representation by aggregating information from its neighboring nodes. This message-passing approach is useful for local graph structure, but it has some limitations. Capturing information from distant nodes requires many layers, which increases computation and may lead to over-smoothing, where node representations become too similar.

Transformers provide an alternative because self-attention allows every token to directly interact with every other token. TokenGT applies this idea to graphs by converting nodes and edges into tokens. Instead of using graph-specific message-passing layers, TokenGT uses a standard Transformer encoder with special token-wise embeddings that encode graph structure.

Our project builds on this idea and asks the following question:

> *Can TokenGT be adapted using a parameter-efficient method such as LoRA, and how does the LoRA rank affect performance?*

To answer this, we modified the original TokenGT codebase by adding LoRA modules to the Transformer attention layers and evaluated the sparse Laplacian-type configuration under multiple LoRA ranks.

2 Background

2.1 Tokenized Graph Transformer

TokenGT treats a graph as a sequence of tokens. For a graph

$$G = (V, E),$$

the nodes in V and edges in E are represented as independent tokens. These tokens are then processed by a Transformer encoder.

A direct Transformer over node and edge tokens is not sufficient because graph structure would be lost. Therefore, TokenGT augments tokens with additional structural embeddings:

- **Node identifiers**: These help the Transformer recognize graph connectivity.

- **Type identifiers**: These tell the model whether a token represents a node or an edge.

In the configuration used in our experiments, we use **Laplacian eigenvector node identifiers** and **type identifiers**. Laplacian eigenvectors act as graph positional encodings and help represent structural positions of nodes in the graph.

2.2 Sparse Laplacian-Type Setup

The base experiment used the sparse Laplacian-type script. The important configuration is summarized below:

Table 1: Baseline sparse Laplacian-type TokenGT configuration.

Parameter	**Value**
Dataset	Synthetic-Barabasi-Albert
Input type	Sparse graph representation
Node identifier	Laplacian eigenvectors
Type identifier	Enabled
Number of layers	2
Hidden dimension	1024
Feed-forward dimension	1024
Query/key dimension	128
Value dimension	128
Batch size	64
Total updates	3000
Warmup updates	1000
Number of graphs	1280
Minimum nodes	10
Maximum nodes	20
Learning rate	1×10^{-4}
End learning rate	1×10^{-9}

The baseline model was trained normally, meaning all model parameters were trainable.

3 LoRA Methodology

3.1 Motivation for LoRA

Full fine-tuning updates all model parameters. In our baseline TokenGT model, this means updating a large Transformer encoder with millions of parameters. This can be expensive in terms of memory, storage, and training time.

Low-Rank Adaptation (LoRA) is a parameter-efficient fine-tuning method. Instead of updating the full weight matrix, LoRA freezes the original weight and learns a low-rank update.

For a linear layer with weight matrix W, the original output is:

$$y = Wx.$$

LoRA modifies this as:

$$y = Wx + \Delta Wx,$$

where

$$\Delta W = BA.$$

Here,

$$A \in \mathbb{R}^{r \times d_{\text{in}}}, \quad B \in \mathbb{R}^{d_{\text{out}} \times r}.$$

The value r is the LoRA rank. A smaller rank means fewer trainable parameters, while a larger rank gives the adapter more capacity.

3.2 LoRA Implementation

We created a new file:

`lora_utils.py`

This file contains the LoRA wrapper module and utility functions. The main module is `LoRALinear`, which wraps an existing `nn.Linear` layer.

The LoRA forward computation is:

$$\text{Output} = \text{BaseLinear}(x) + \frac{\alpha}{r} B(A(x)).$$

The original base linear layer is frozen, and only the LoRA matrices A and B are trained.

3.3 Layers Modified

We applied LoRA to the attention-related linear layers of the Transformer encoder. Specifically, LoRA was inserted into:

- `attn.fc1`

- `attn.fc_v`
- `attn.fc_out`

For the `n_layers=2` configuration, the logs showed that LoRA was applied to attention layers across the indexed Transformer blocks:

- `layers.0.attn.fc1`, `layers.0.attn.fc_v`, `layers.0.attn.fc_out`
- `layers.1.attn.fc1`, `layers.1.attn.fc_v`, `layers.1.attn.fc_out`
- `layers.2.attn.fc1`, `layers.2.attn.fc_v`, `layers.2.attn.fc_out`

This means the experiment performs attention-only LoRA adaptation. We did not apply LoRA to the feed-forward network layers in the final reported experiments.

3.4 Files Modified

The implementation required the following code-level changes:

1. **Created `lora_utils.py`:** Added `LoRALinear`, `apply_lora_to_model`, `freeze_non_lora_params`, and parameter counting utilities.
2. **Modified `train.py`:** Added LoRA-related arguments and logic to apply LoRA inside the model. The optimizer was changed to update only trainable parameters.
3. **Modified `entry.py`:** Added LoRA command-line arguments and checkpoint-loading support for LoRA runs.
4. **Added new shell scripts:** Created separate training and testing scripts for LoRA ranks 4, 16, 64, 128, and 256.

4 Experimental Setup

4.1 Baseline Experiment

The baseline experiment used the original sparse Laplacian-type TokenGT setup. It was run using:

```
sparse-lap-type.sh
```

The baseline was evaluated using:

```
sparse-lap-type-test.sh
```

The baseline achieved:

$$\textbf{Test Loss} = 0.017566.$$

This is the best result among all runs because the baseline trains the full model.

4.2 LoRA Experiments

For LoRA, we created separate scripts for different ranks:

- `sparse-lap-type-lora-r4.sh`
- `sparse-lap-type-lora-r16.sh`
- `sparse-lap-type-lora-r64.sh`
- `sparse-lap-type-lora-r128.sh`
- `sparse-lap-type-lora-r256.sh`

Each script uses the same dataset and sparse Laplacian-type TokenGT configuration. The main difference is the LoRA rank and LoRA alpha.

The general relation used was:

$$\alpha = 2r.$$

For example:

$$r = 16 \Rightarrow \alpha = 32,$$

$$r = 256 \Rightarrow \alpha = 512.$$

The LoRA dropout used in the training scripts was:

$$\text{LoRA dropout} = 0.05.$$

The LoRA target was:

$$\texttt{lora\_target = attn}.$$

This means only attention layers were adapted.

5 Results

5.1 Main Result Table

The test losses obtained from the baseline and LoRA experiments are shown in Table 2.

Table 2: Baseline vs LoRA rank comparison. Lower test loss is better.

Model	LoRA Rank	LoRA Alpha	Target Layers	Test Loss
Full TokenGT baseline	–	–	Full model	**0.017566**
LoRA-r4	4	8	Attention only	1.340000
LoRA-r16	16	32	Attention only	0.240000
LoRA-r64	64	128	Attention only	0.107000
LoRA-r128	128	256	Attention only	0.080000
LoRA-r256	256	512	Attention only	0.060000

5.2 Improvement Across LoRA Ranks

To better understand the effect of increasing rank, Table 3 compares LoRA losses with the rank-4 result and the full baseline.

Table 3: Relative comparison of LoRA performance.

Model	Test Loss	Reduction vs r4	Loss / Baseline Loss
LoRA-r4	1.340000	0.00%	76.28×
LoRA-r16	0.240000	82.09%	13.66×
LoRA-r64	0.107000	92.01%	6.09×
LoRA-r128	0.080000	94.03%	4.55×
LoRA-r256	0.060000	95.52%	3.42×

5.3 Observed Trend

The results show a clear pattern:

$$\text{LoRA-r4} > \text{LoRA-r16} > \text{LoRA-r64} > \text{LoRA-r128} > \text{LoRA-r256}$$

in terms of loss, where lower is better. Therefore, increasing LoRA rank consistently improved the result.

The best LoRA result was obtained at rank 256:

$$\text{LoRA-r256 test loss} = 0.06.$$

However, the full baseline was still better:

$$\text{Baseline test loss} = 0.017566.$$

This shows that LoRA was able to recover a significant portion of the baseline performance, but it did not completely match full fine-tuning.

6 Analysis and Discussion

6.1 Why Does Higher Rank Improve LoRA Performance?

The LoRA rank controls the capacity of the adapter. A low rank forces the weight update to lie in a very small subspace. This is parameter-efficient, but it may not be expressive enough to capture all changes needed by the task.

At rank 4, the model had very limited adaptation capacity. This explains why the loss was high:

$$\text{LoRA-r4 loss} = 1.34.$$

When the rank was increased to 16, the model gained more trainable degrees of freedom:

$$\text{LoRA-r16 loss} = 0.24.$$

The improvement continued as rank increased to 64, 128, and 256. This suggests that the task requires a fairly expressive update to the attention layers.

6.2 Why Did LoRA Not Beat the Baseline?

The full baseline updates all trainable parameters in the TokenGT model. In contrast, LoRA freezes most of the model and only trains low-rank adapter matrices inside selected attention layers.

This creates an important trade-off:

- **Baseline:** Higher performance, but many trainable parameters.
- **LoRA:** Fewer trainable parameters, but slightly worse performance.

The baseline achieved a very low loss of 0.017566. Since this is already strong performance, it is difficult for a parameter-efficient method to outperform it, especially when LoRA is applied only to attention layers.

Another reason is that we used attention-only LoRA. The feed-forward network layers were not adapted. In Transformer models, the feed-forward blocks also contain a large

amount of task-specific capacity. Since these layers were frozen, the LoRA model had less flexibility than full fine-tuning.

6.3 Interpretation of Rank 256 Result

The rank-256 result is important because it shows that LoRA can approach the baseline more closely when given enough rank. The test loss decreased to 0.06, which is much better than rank 4 and rank 16.

However, increasing rank also increases the number of trainable parameters. Therefore, rank 256 is less parameter-efficient than rank 4 or rank 16. This means that the best rank depends on the objective:

- If the goal is maximum parameter efficiency, rank 4 or rank 16 is more attractive.
- If the goal is better performance while still training fewer parameters than full fine-tuning, rank 128 or rank 256 is more useful.

6.4 Practical Meaning of the Results

The results are meaningful because they show a smooth improvement as rank increases. This indicates that the LoRA implementation is learning useful adaptations and that rank is an important hyperparameter.

The overall conclusion is not that LoRA beats full fine-tuning. Instead, the conclusion is:

> *LoRA provides a controllable trade-off between parameter efficiency and performance in TokenGT. Increasing the rank improves performance, but full-parameter training remains the strongest configuration for this task.*

7 Possible Hyperparameter Extensions

Due to time and system constraints, we mainly tested the effect of LoRA rank. However, several other hyperparameters can be modified in future experiments.

7.1 Changing LoRA Target Layers

In our current runs, we used:

$$\texttt{lora\_target = attn}.$$

This means LoRA was applied only to attention layers. A natural extension is:

$$\texttt{lora\_target} = \texttt{all}.$$

This would apply LoRA to both attention and feed-forward network layers. Since feed-forward layers contain significant model capacity, adapting them may reduce the gap between LoRA and the full baseline.

Expected effect:

- More trainable parameters.
- Better performance.
- Higher memory and computation cost.

7.2 Changing LoRA Dropout

We used:

$$\texttt{lora\_dropout} = 0.05.$$

Dropout can help generalization, but for a small synthetic task it may reduce the model's ability to fit the data. A useful future experiment is:

$$\texttt{lora\_dropout} = 0.0.$$

Expected effect:

- Faster fitting.
- Possibly lower test loss.
- Slightly higher risk of overfitting.

7.3 Changing Learning Rate

We used:

$$\texttt{peak\_lr} = 1 \times 10^{-4}.$$

LoRA often benefits from a slightly larger learning rate because only a small number of parameters are being updated. Future experiments can try:

$$3 \times 10^{-4}, \quad 5 \times 10^{-4}, \quad 1 \times 10^{-3}.$$

Expected effect:

- Higher learning rate may improve LoRA adaptation.
- Too high a learning rate may destabilize training.
- The best learning rate may depend on the LoRA rank.

7.4 Changing Number of Training Updates

All main experiments used:

$$\texttt{tot\_updates} = 3000.$$

Higher-rank LoRA has more trainable parameters and may require more optimization steps. Future work can test:

$$5000, \quad 8000, \quad 10000$$

training updates.

Expected effect:

- Higher ranks may improve with longer training.
- Training time will increase.
- Better convergence may be observed for rank 128 and rank 256.

7.5 Changing Batch Size

The training scripts used batch size 64, while some test scripts used batch size 16. Since evaluation batch size does not change the mathematical model, this mostly affects memory usage and evaluation speed. For training, batch size can affect stability and convergence.

Future experiments can compare:

$$16, \quad 32, \quad 64, \quad 128.$$

Expected effect:

- Larger batch size may stabilize gradients.
- Smaller batch size may introduce useful noise.
- GPU memory limits may restrict the maximum batch size.

7.6 Changing Number of Layers

The current experiments used:

$$\texttt{n\_layers} = 2.$$

A deeper TokenGT model may improve performance but will increase memory and training time. Future experiments can try:

$$\texttt{n\_layers} = 3, 4, 6.$$

Expected effect:

- More layers may improve expressiveness.
- More layers may require stronger regularization.
- LoRA may become more useful in deeper models because there are more layers to adapt.

7.7 Summary of Future Hyperparameters

Table 4: Future hyperparameter directions and expected effects.

Hyperparameter	**Possible Values**	**Expected Effect**
LoRA target	Attention, all layers	Applying LoRA to FFN layers may improve performance.
LoRA dropout	0.0, 0.05, 0.1	Lower dropout may fit better; higher dropout may regularize.
Peak learning rate	$1e^{-4}$, $3e^{-4}$, $5e^{-4}$	Higher LR may help LoRA learn faster.
Training updates	3000, 5000, 10000	More updates may improve high-rank LoRA.
Batch size	16, 32, 64, 128	Affects stability, speed, and GPU memory.
LoRA rank	4, 16, 64, 128, 256, 512	Higher rank improves capacity but reduces parameter efficiency.
Number of layers	2, 3, 4, 6	Deeper models may improve performance but cost more.

8 Limitations

Although our experiments show a clear trend, there are some limitations.

1. **Limited hyperparameter search:** We mainly varied LoRA rank. Other hyperparameters such as learning rate, dropout, target layers, and number of training updates were not exhaustively tested.

2. **Attention-only LoRA:** We applied LoRA only to attention layers. The feed-forward layers remained frozen, which may limit adaptation capacity.

3. **System constraints:** Some runs were limited by available disk space, GPU availability, and total training time. Because of this, we could not run a broader grid search.

4. **Synthetic dataset:** The experiments were conducted on the Synthetic-Barabasi-Albert setup. Results may differ on larger real-world graph datasets.

5. **Minor script differences:** Some scripts had small differences in batch size, dropout, and checkpoint names. We tried to keep the main training setup consistent, but a stricter future study should standardize every script exactly.

9 Conclusion

In this project, we modified the TokenGT implementation by adding LoRA-based parameter-efficient adaptation. We created a new LoRA utility module, modified the training pipeline, and wrote separate scripts for multiple LoRA ranks.

The main finding is that LoRA rank strongly affects performance. The loss decreased consistently as rank increased:

$$1.34 \to 0.24 \to 0.107 \to 0.08 \to 0.06.$$

This shows that higher-rank LoRA adapters have more capacity to approximate the updates learned during full fine-tuning.

However, the full baseline still achieved the best result:

$$\text{Baseline loss} = 0.017566.$$

Therefore, LoRA did not outperform full fine-tuning in our current setup. Instead, LoRA provided a useful parameter-efficient alternative that reduced the performance gap as rank increased.

Due to time and system constraints, we could not test all possible combinations of learning rate, dropout, target layers, training updates, and deeper architectures. Future work should apply LoRA to both attention and feed-forward layers, tune learning rate, reduce LoRA dropout, and test longer training for high-rank adapters. These changes may further reduce the gap between LoRA and full fine-tuning.

References

[1] J. Kim, T. D. Nguyen, S. Min, S. Cho, M. Lee, H. Lee, and S. Hong, *Pure Transformers are Powerful Graph Learners*, NeurIPS, 2022.

[2] E. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, *LoRA: Low-Rank Adaptation of Large Language Models*, ICLR, 2022.

[3] A. Vaswani et al., *Attention Is All You Need*, NeurIPS, 2017.

[4] T. Kipf and M. Welling, *Semi-Supervised Classification with Graph Convolutional Networks*, ICLR, 2017.

[5] P. Veličković et al., *Graph Attention Networks*, ICLR, 2018.